ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Доцент, канд. тех. наук		Степанов П. А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Разработка ресурса REST/JSON сервиса

по курсу: Технология разработки серверных информационных систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4434		Энгельгардт К. А.
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Текст и вариант задания

Вариант задания: 1 - Книжный магазин либо библиотека.

Задание на лабораторную работу:

- Определить перечень REST-сервисов, выполняющих те же действия, что и в лабораторной работе 1. Внимательно отнестись к вопросу, какой HTTP-метод использует тот или иной сервис и какие коды HTTP он может возвращать.
- Описать API с помощью OpenAPI v3. Привести YAML-файл в отчёте.
- API должен содержать все четыре метода CRUD - добавление, удаление, изменение и чтение. Следить за тем, чтобы API оперировал только JSON-представлениями и не возвращал в качестве результатов строки с сообщениями.
- Средствами Swagger сгенерировать сервер Spring (необязательно).
- Написать REST-контроллер самостоятельно (пункт 4 не выполнялся).
- Реализовать приложение. Swagger UI должен быть доступен через web по адресу http://localhost:8080/swagger-ui/index.html.

2. Описание разрабатываемого продукта

В рамках лабораторной работы разработан RESTful веб-сервис «Книжный магазин» на базе фреймворка Spring Boot 3.2.5. Сервис реализует REST/JSON API для управления каталогом книг и обеспечивает выполнение полного набора CRUD-операций: создание, чтение, обновление и удаление.

Для документирования и тестирования API использована библиотека SpringDoc OpenAPI, которая автоматически формирует интерактивный интерфейс Swagger UI на основе аннотаций в исходном коде. Данные хранятся в оперативной памяти (in-memory хранилище) без подключения к базе данных.

2.1. Архитектура приложения

Приложение построено по слоистой (layered) архитектуре и включает следующие компоненты:

Модель (Model): Book.java - POJO-класс, описывающий сущность книги с полями: id, title, author, genre, year, price, stock.

Хранилище (Repository): BookRepository.java - in-memory хранилище на основе LinkedHashMap с атомарным счётчиком идентификаторов. Содержит методы save, findAll, findById, update, deleteById.

Сервис (Service): BookService.java - слой бизнес-логики, делегирующий операции в репозиторий.

Контроллер (Controller): BookRestController.java - REST-контроллер, реализующий HTTP-эндпоинты и возвращающий объекты ResponseEntity<Book> с корректными HTTP-статусами.

Конфигурация (Config): OpenApiConfig.java - настройка метаданных OpenAPI (название, описание, контакт, лицензия).

2.2. Стек технологий

- Java 17 (Oracle HotSpot JDK 17.0.12)
- Spring Boot 3.2.5 (spring-boot-starter-web, встроенный Apache Tomcat 10)
- SpringDoc OpenAPI 2.5.0 (генерация Swagger UI и OpenAPI v3 описания)
- Lombok (сокращение шаблонного кода: @Data, @Builder, @RequiredArgsConstructor)
- Apache Maven 3.9.16 (система сборки)

2.3. Описание REST API

Все эндпоинты API расположены по базовому пути /api/books. Формат данных - исключительно application/json. В таблице 1 представлен полный перечень доступных операций.

Таблица 1 - Перечень REST-сервисов книжного магазина

HTTP-метод	URL	Действие	Коды ответа
POST	/api/books	Создать книгу	201 Created 400 Bad Request
GET	/api/books	Список всех книг	200 OK
GET	/api/books/{id}	Получить книгу по ID	200 OK 404 Not Found
PUT	/api/books/{id}	Обновить книгу	200 OK 404 Not Found
DELETE	/api/books/{id}	Удалить книгу	204 No Content 404 Not Found

Согласно принципам REST, HTTP-метод определяет семантику операции: POST - создание нового ресурса (код 201 Created), GET - безопасное чтение без изменения состояния (200 OK), PUT - полное замещение ресурса (200 OK), DELETE - удаление ресурса (204 No Content). В случае отсутствия запрашиваемого ресурса возвращается код 404 Not Found. API не возвращает строковые сообщения в качестве тела ответа - только JSON-объекты или пустое тело.

3. Текст основных фрагментов кода

3.1. Модель Book.java

Класс Book является POJO-объектом (Plain Old Java Object), описывающим книгу. Аннотации Lombok @Data и @Builder автоматически генерируют геттеры, сеттеры, equals,

hashCode и конструкторы. Аннотации @Schema из библиотеки Swagger обеспечивают документирование полей в Swagger UI.

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Schema(description = "Модель книги")
public class Book {

    @Schema(description = "Уникальный идентификатор книги",
example = "1")
    private Long id;

    @Schema(description = "Название книги", example = "Война и
мир")
    private String title;

    @Schema(description = "Автор книги", example = "Лев Толстой")
    private String author;

    @Schema(description = "Жанр книги", example = "Роман")
    private String genre;

    @Schema(description = "Год издания", example = "1869")
    private Integer year;

    @Schema(description = "Цена книги в рублях", example =
"590.00")
    private Double price;

    @Schema(description = "Количество экземпляров на складе",
example = "10")
    private Integer stock;
}
```

3.2. Хранилище BookRepository.java

Данные хранятся в LinkedHashMap - упорядоченном словаре, ключами которого служат идентификаторы книг. Идентификаторы назначаются с помощью AtomicLong для потокобезопасности. При старте приложения репозиторий инициализируется тремя тестовыми записями.

```
@Repository
```

```
public class BookRepository {

    private final Map<Long, Book> store = new LinkedHashMap<>();
    private final AtomicLong idSequence = new AtomicLong(1);

    public Book save(Book book) {
        book.setId(idSequence.getAndIncrement());
        store.put(book.getId(), book);
        return book;
    }

    public List<Book> findAll() {
        return new ArrayList<>(store.values());
    }

    public Optional<Book> findById(Long id) {
        return Optional.ofNullable(store.get(id));
    }

    public Optional<Book> update(Long id, Book updated) {
        if (!store.containsKey(id)) return Optional.empty();
        updated.setId(id);
        store.put(id, updated);
        return Optional.of(updated);
    }

    public boolean deleteById(Long id) {
        return store.remove(id) != null;
    }
}
```

3.3. REST-контроллер BookRestController.java

Контроллер аннотирован @RestController и @RequestMapping("/api/books"), что задаёт базовый путь для всех эндпоинтов. Каждый метод аннотирован соответствующим HTTP-методом (@PostMapping, @GetMapping, @PutMapping, @DeleteMapping) и аннотациями OpenAPI для документирования. Методы возвращают ResponseEntity, позволяя явно задавать HTTP-статус ответа.

```
@RestController
@RequestMapping("/api/books")
@RequiredArgsConstructor
```

```
@Tag(name = "Книги", description = "API каталога книжного
магазина")
public class BookRestController {

    private final BookService bookService;

    // POST /api/books - Создать книгу (201 Created)
    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book
book) {
        Book created = bookService.create(book);
        return
ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    // GET /api/books - Список всех книг (200 OK)
    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAll());
    }

    // GET /api/books/{id} - Книга по ID (200 OK / 404 Not Found)
    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long
id) {
        return bookService.getById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // PUT /api/books/{id} - Обновить книгу (200 OK / 404 Not
Found)
    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(@PathVariable Long id,
                                           @RequestBody Book
book) {
        return bookService.update(id, book)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // DELETE /api/books/{id} - Удалить книгу (204 No Content /
404 Not Found)
```

```
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id)
{
        if (bookService.delete(id)) {
            return ResponseEntity.noContent().build();
        }
        return ResponseEntity.notFound().build();
    }
}
```

3.4. Описание API в формате OpenAPI v3 (openapi.yaml)

В соответствии с заданием разработан YAML-файл в формате OpenAPI v3, формально описывающий все эндпоинты сервиса, форматы запросов и ответов, а также возможные HTTP-коды. Ниже приведён фрагмент описания.

```
openapi: 3.0.1
info:
  title: Книжный магазин - REST API
  description: Лабораторная работа №2. CRUD-сервис каталога
книжного магазина.
  version: 1.0.0

paths:
  /api/books:
    post:
      summary: Добавить книгу
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Book'
      responses:
        '201': { description: Книга успешно создана }
        '400': { description: Некорректные входные данные }
    get:
      summary: Получить все книги
      responses:
        '200':
          content:
            application/json:
              schema:
```

```
                type: array
                items: { $ref: '#/components/schemas/Book' }

  /api/books/{id}:
    get:
      summary: Получить книгу по ID
      parameters:
        - { name: id, in: path, required: true, schema: { type:
integer } }
      responses:
        '200': { description: Книга найдена }
        '404': { description: Книга не найдена }
    put:
      summary: Обновить книгу
      responses:
        '200': { description: Книга обновлена }
        '404': { description: Книга не найдена }
    delete:
      summary: Удалить книгу
      responses:
        '204': { description: Книга удалена }
        '404': { description: Книга не найдена }

components:
  schemas:
    Book:
      type: object
      properties:
        id:     { type: integer, format: int64, readOnly: true }
        title:  { type: string,  example: "Война и мир" }
        author: { type: string,  example: "Лев Толстой" }
        genre:  { type: string,  example: "Роман" }
        year:   { type: integer, example: 1869 }
        price:  { type: number,  format: double, example: 590.00
}
        stock:  { type: integer, example: 10 }
```

3.5. Конфигурация сборки pom.xml (фрагмент)

Файл pom.xml описывает зависимости проекта. Ключевые зависимости: spring-boot-starter-web (REST + встроенный Tomcat), springdoc-openapi-starter-webmvc-ui (Swagger UI / OpenAPI 3), lombok (генерация шаблонного кода).

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
</parent>

<dependencies>
    <!-- Spring Boot Web (REST + Embedded Tomcat) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- OpenAPI 3 / Swagger UI -->
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-
ui</artifactId>
        <version>2.5.0</version>
    </dependency>

    <!-- Lombok -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
```

Заключение

В ходе выполнения лабораторной работы №2 был разработан RESTful веб-сервис «Книжный магазин» на базе Spring Boot. Реализован полный набор CRUD-операций (создание, чтение, обновление, удаление) в соответствии с принципами REST. API оперирует исключительно JSON-форматом и возвращает семантически корректные HTTP-коды ответа. Разработано формальное описание интерфейса в формате OpenAPI v3. Интерактивный Swagger UI доступен по адресу http://localhost:8080/swagger-ui/index.html и позволяет тестировать все эндпоинты непосредственно из браузера.